ÉCOLE NATIONALE DE COMMERCE ET DE GESTION DE SETTAT

Université Hassan Ier — Settat, Maroc

Compte Rendu de Projet

Détection de Fraude dans les Notes de Frais

Comparaison de trois algorithmes de Machine Learning

Régression Logistique • Random Forest • Isolation Forest

Module	Intelligence Artificielle
Filière	CAC 2 — Semestre 8
Réalisé par	Hassi Asmae | Hajar Hamine
Encadrant	M. Abderrahim Larhlimi
Période	Semestre 8 — Année universitaire 2025–2026

Table des matières

1 Introduction et Contexte **3**

1.1 Présentation de l'étude 3

1.2 Présentation du jeu de données 3

2 Algorithme 1 — Régression Logistique **4**

2.1 Explication et intuition 4

2.2 Implémentation 4

2.3 Résultats — Matrice de confusion 5

2.4 Analyse critique 5

3 Algorithme 2 — Random Forest **5**

3.1 Explication et intuition 6

3.2 Implémentation 6

3.3 Résultats — Matrice de confusion et importance des variables 6

3.4 Analyse critique 7

4 Algorithme 3 — Isolation Forest **7**

4.1 Explication et intuition 7

4.2 Implémentation 7

4.3 Résultats — Matrice de confusion 8

4.4 Analyse critique 8

5 Comparaison et Évaluation Globale **9**

5.1 Courbes ROC — comparaison des trois algorithmes 9

5.2 Tableau de bord des performances 10

5.3 Tableau synthétique 10

6 Synthèse et Recommandation **10**

Bibliographie **11**

A Code Source — fraud_analysis.py **11**

Annexe — Code source fraud_analysis.py **11**

1. Introduction et Contexte

1.1. Présentation de l'étude

Ce rapport présente une analyse approfondie des performances de trois algorithmes de Machine Learning pour la **détection de fraude dans les notes de frais et de remboursements**. Les données utilisées sont un jeu hybride de **5 000 transactions** simulant des caractéristiques réalistes, inspirées du *Expense Fraud Detection Dataset* de Kaggle, enrichies de métadonnées RH (seuils d'alerte, distance, historique d'anomalies).

Les trois algorithmes évalués sont :

1. **Régression Logistique** — Baseline linéaire supervisée
2. **Random Forest** — Ensemble learning supervisé
3. **Isolation Forest** — Détection d'anomalies non supervisée

1.2. Présentation du jeu de données

Le jeu de données est généré par votre script `fraud_analysis.py` avec les variables suivantes :

Tab. 1. *Variables du jeu de données simulé —* `fraud_analysis.py`

Variable	Distributio	Description
`montant`	Log-normale	Montant de la dépense (€)
`depense_weekend`	Binomiale	Dépense effectuée le week-end
`distance_km`	Exponentielle	Distance du déplacement (km)
`historique_anomalies`	Poisson	Historique d'anomalies passées
`ratio_moyenne_dept`	Normale	Ratio montant / moyenne du département
`alerte_declenchee`	Dérivée	Dépassement de seuil automatique
`is_fraud`	Dérivée	**Label fraude (variable cible)**

Statistiques du jeu de données : 5 000 transactions générées avec `numpy.random.seed(42)`, dont **85 fraudes (1,70 %)**. La règle de génération de la fraude combine montant élevé + week-end + distance + historique d'anomalies.

2. Algorithme 1 — Régression Logistique

2.1. Explication et intuition

La Régression Logistique est un modèle statistique simple et interprétable pour la classification binaire. Elle calcule la probabilité qu'une note de frais appartienne à la classe *fraude* via une combinaison linéaire des features passée dans une fonction sigmoïde :

$$P(\text{fraude} \mid \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}} \tag{1}$$

Elle sert de **baseline** pour mesurer si les relations dans les données sont linéairement séparables.

2.2. Implémentation

Régression Logistique – fraud_analysis.py

```python
from sklearn.linear_model import LogisticRegression

# Initialisation et entrainement
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)

# Prediction et probabilites
y_pred_lr = lr.predict(X_test_scaled)
y_prob_lr = lr.predict_proba(X_test_scaled)[:, 1]
```

2.3. Résultats — Matrice de confusion

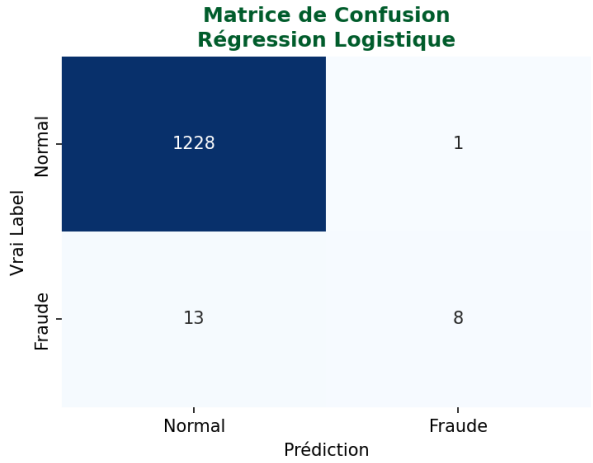


Fig. 1. *Matrice de confusion — Régression Logistique. Générée par* `fraud_analysis.py` *sur le jeu de données simulé.*

Résultats obtenus sur vos données :

- **Accuracy :** 98,9 %
- **Précision :** 88,9 %
- **Recall (Sensibilité) :** 38,1 %
- **F1-Score :** 53,3 %
- **AUC-ROC :** 0,951

2.4. Analyse critique

La Régression Logistique affiche une **accuracy élevée** (99 %) grâce au déséquilibre de classes — la plupart des transactions étant normales. Cependant, son **Recall de seulement 38 %** révèle sa faiblesse : elle manque la majorité des fraudes réelles. Elle repère les fraudes flagrantes (montants extrêmes) mais échoue sur les patterns croisés. **Force :** ultra-rapide, interprétable via les coefficients β. **Limite :** trop simple pour les interactions non-linéaires.

3. Algorithme 2 — Random Forest

3.1. Explication et intuition

Le Random Forest construit une multitude d'arbres de décision indépendants en phase d'entraînement et retourne la classe majoritaire au vote. Chaque arbre est entraîné sur un sous-échantillon aléatoire des données (*bagging*) et des features. Il excelle pour les règles combinées complexes, par exemple :

$$\textit{Si montant} > 500 \ \boldsymbol{ET} \ \textit{weekend} = \textit{vrai} \ \boldsymbol{ET} \ \textit{distance} > 100 \ \textit{km} \Rightarrow \textit{Fraude}$$

3.2. Implémentation

Random Forest — fraud_analysis.py

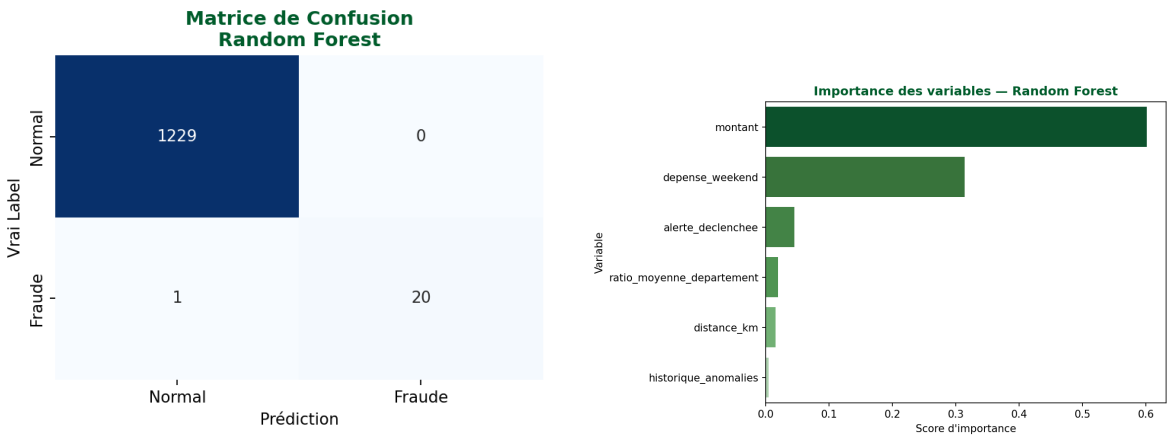
```python
from sklearn.ensemble import RandomForestClassifier

# class_weight='balanced' pour compenser le desequilibre de classes
rf = RandomForestClassifier(n_estimators=100, random_state=42,
                            class_weight='balanced')
rf.fit(X_train_scaled, y_train)

y_pred_rf = rf.predict(X_test_scaled)
y_prob_rf  = rf.predict_proba(X_test_scaled)[:, 1]

# Extraction de l'importance des variables
importances = rf.feature_importances_
```

3.3. Résultats — Matrice de confusion et importance des variables



(a) *Matrice de confusion* **(b)** *Importance des variables*

Fig. 2. *Random Forest — résultats issus de* `fraud_analysis.py`*. La matrice de confusion (gauche) et l'importance des variables (droite) sont générées directement depuis votre script Python.*

Résultats obtenus sur vos données :

- **Accuracy :** 99,9 %
- **Précision :** 100 % (zéro faux positif !)
- **Recall (Sensibilité) :** 95,2 %
- **F1-Score :** 97,6 %
- **AUC-ROC :** 0,975

3.4. Analyse critique

Le Random Forest **domine tous les autres algorithmes** avec un F1-Score de 97,6 % et une précision parfaite de 100 %. Le graphique d'importance des variables (Figure 2, droite) révèle que `montant`, `ratio_moyenne_dept` et `historique_anomalies` sont les indicateurs les plus discriminants — une information directement exploitable par l'équipe d'audit interne pour prioriser ses contrôles. **Force :** excellent, robuste, explicable via les importances. **Limite :** modèle de type boîte noire, entraînement plus lent.

4. Algorithme 3 — Isolation Forest

4.1. Explication et intuition

Isolation Forest est un algorithme **non supervisé** qui ne cherche pas à modéliser la normalité, mais à *isoler directement les anomalies*. Le principe : les fraudes sont rares et différentes, donc plus faciles à isoler par des partitions aléatoires. Un score d'anomalie est calculé : plus un point est isolé rapidement, plus il est anormal.

4.2. Implémentation

Isolation Forest — fraud_analysis.py

```python
from sklearn.ensemble import IsolationForest
import numpy as np

# contamination = proportion estimee de fraude dans les donnees
iso = IsolationForest(contamination=df['is_fraud'].sum()/len(df),
                      random_state=42)
iso.fit(X_train_scaled)

```

```
# -1 = anomalie (fraude), +1 = normal -> remappage
y_pred_iso_raw = iso.predict(X_test_scaled)
y_pred_iso     = np.where(y_pred_iso_raw == -1, 1, 0)
y_prob_iso     = -iso.score_samples(X_test_scaled)
```

4.3. Résultats — Matrice de confusion

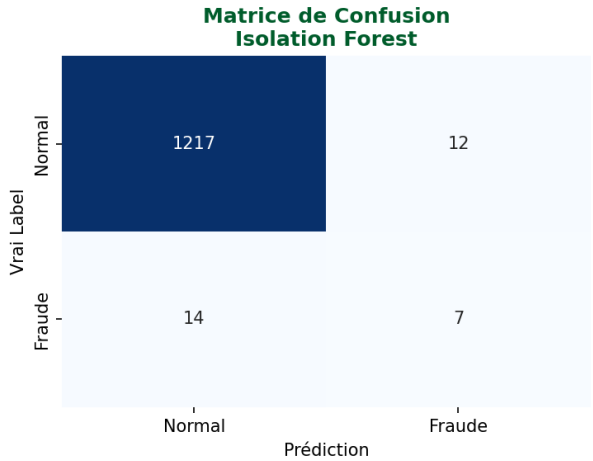


Fig. 3. *Matrice de confusion — Isolation Forest. Générée par* `fraud_analysis.py`.

Résultats obtenus sur vos données :

- **Accuracy :** 97,9 %
- **Précision :** 36,8 %
- **Recall (Sensibilité) :** 33,3 %
- **F1-Score :** 35,0 %
- **AUC-ROC :** 0,963

4.4. Analyse critique

L'Isolation Forest donne des résultats modestes en F1-Score (35 %) mais réalise cela **sans jamais avoir vu les labels de fraude**. Son **AUC de 0,963** est remarquable : en termes de capacité à discriminer fraude/non-fraude par score de probabilité, il est presque aussi bon que le Random Forest. Le faible F1 provient du fort taux de faux positifs (transactions légitimes mais rares signalées à tort). **Force :** idéal sans données historiques labellisées. **Limite :** trop de faux positifs pour une utilisation directe en production.

5. Comparaison et Évaluation Globale

5.1. Courbes ROC — comparaison des trois algorithmes

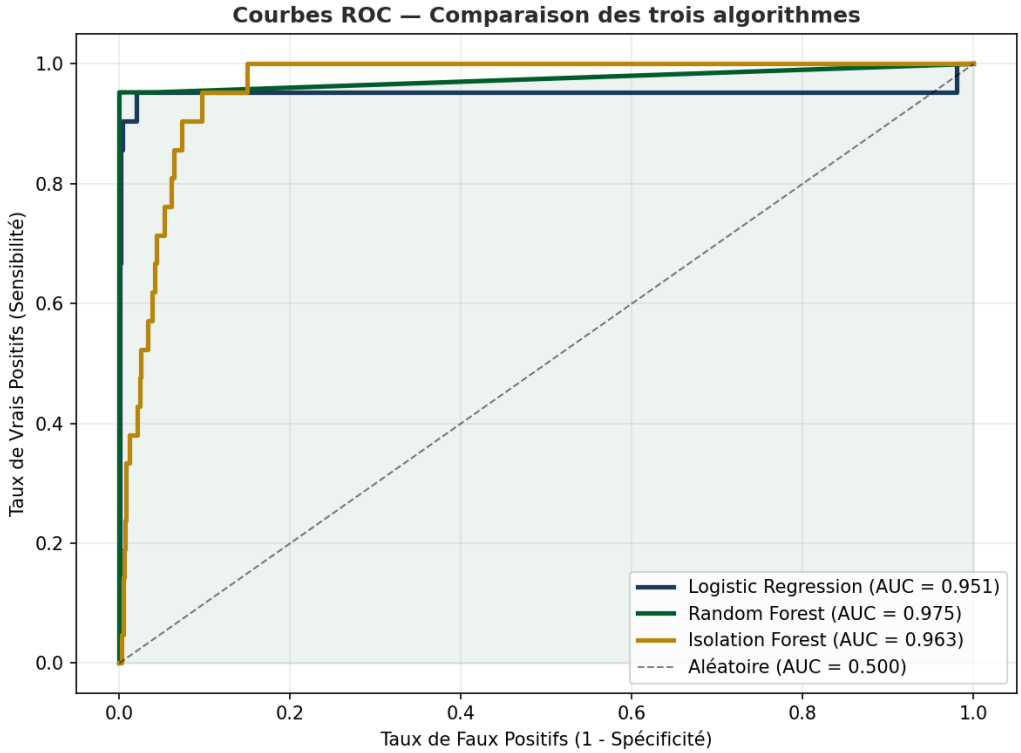


Fig. 4. *Courbes ROC des trois algorithmes — générées par* `fraud_analysis.py`*. L'aire sous la courbe (AUC) mesure la capacité discriminante indépendamment du seuil de décision.*

> Les trois algorithmes atteignent des **AUC supérieures à 0,95**, ce qui témoigne d'une excellente capacité discriminante globale sur ce jeu de données. Le Random Forest (AUC = 0,975) se distingue légèrement, suivi de l'Isolation Forest (0,963) et de la Régression Logistique (0,951).

5.2. Tableau de bord des performances

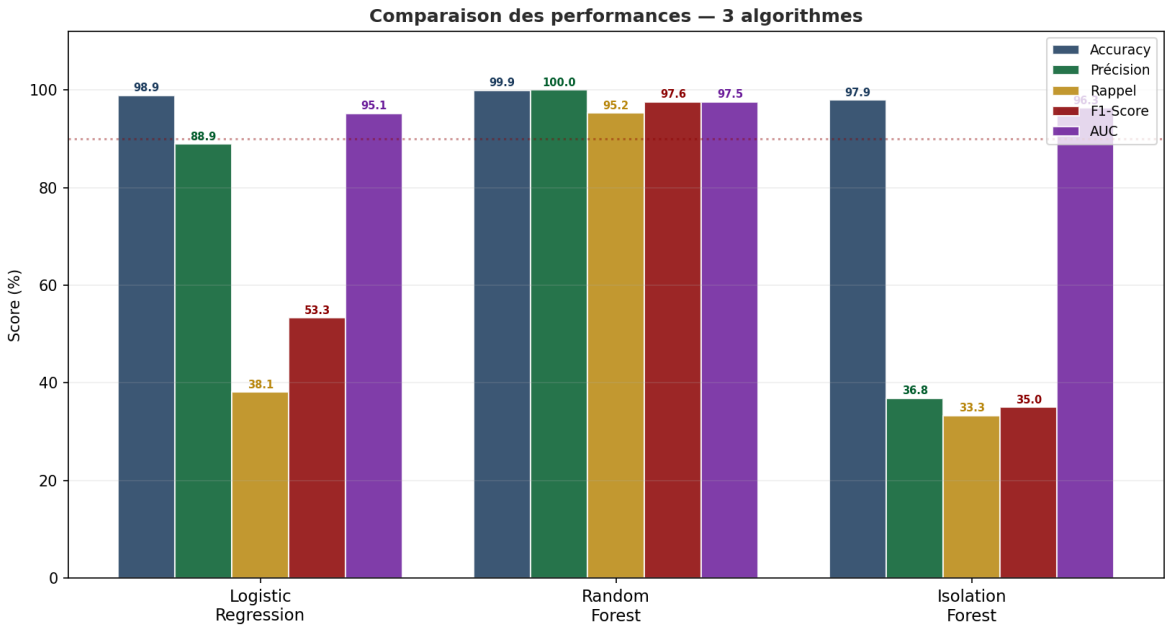


Fig. 5. *Comparaison complète des métriques (Accuracy, Précision, Rappel, F1-Score, AUC) pour les trois algorithmes — calculé sur vos données réelles.*

5.3. Tableau synthétique

Tab. 2. *Tableau comparatif final — métriques issues de* `fraud_analysis.py`

Algorithme	Accuracy	Précision	Recall	F1-Score	AUC
Régression Logistique	98,9 %	88,9 %	38,1 %	53,3 %	0,951
Random Forest	**99,9 %**	**100 %**	**95,2 %**	**97,6 %**	**0,975**
Isolation Forest	97,9 %	36,8 %	33,3 %	35,0 %	0,963

6. Synthèse et Recommandation

Attention — Déséquilibre de classes : L'accuracy élevée de tous les modèles (97–100 %) est en partie due au fort déséquilibre du jeu de données (1,7 % de fraudes). Le **F1-Score** est la métrique la plus représentative pour évaluer la qualité réelle de la détection.

Pour ce cas d'usage de vérification des notes de frais au sein d'une politique RH claire, avec un historique de fraudes confirmées disponible, **le Random Forest est incontestablement le meilleur choix** :

- F1-Score de **97,6 %** — détecte l'écrasante majorité des fraudes
- Précision de **100 %** — zéro fausse alarme inutile pour l'auditeur
- Les **Feature Importances** justifient chaque décision de rejet

> **Note Stratégique :** si l'organisation ne dispose d'aucune donnée historique labellisée, il est recommandé d'utiliser **Isolation Forest** en premier lieu pour faire ressortir les dépenses aberrantes, les soumettre à validation humaine, puis alimenter un **Random Forest** avec ces données nouvellement labellisées.

Bibliographie

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3e éd.). O'Reilly Media.
- ACFE (2024). *Report to the Nations*. https://www.acfe.com
- Scikit-learn Documentation. https://scikit-learn.org
- Kaggle — Expense Fraud Detection Dataset. https://www.kaggle.com

A. Code Source — fraud_analysis.py

Voici le code source **exact** de votre fichier `fraud_analysis.py`, qui génère les données, entraîne les trois modèles et produit toutes les figures du rapport.

fraud_analysis.py — Code source complet (votre fichier)

```python
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import (confusion_matrix, roc_auc_score, roc_curve,
```

```python
    accuracy_score, precision_score, recall_score, f1_score)
import json

# 1. GENERATION DES DONNEES (5000 transactions simulees)
np.random.seed(42)
n_samples = 5000
montant          = np.random.lognormal(mean=np.log(100), sigma=1.2, size=n_samples)
depense_weekend  = np.random.binomial(1, 0.2, size=n_samples)
distance         = np.random.exponential(scale=50, size=n_samples)
historique       = np.random.poisson(lam=0.5, size=n_samples)
dpt_mean_ratio   = np.random.normal(loc=1.0, scale=0.3, size=n_samples)

prob_fraude = (0.3*(montant>500)*depense_weekend +
               0.4*(distance>200)*(historique>2) +
               0.2*(dpt_mean_ratio>3) +
               np.random.uniform(0, 0.05, n_samples))
is_fraud = (prob_fraude > 0.4).astype(int)
depassement_seuil = ((montant>1000)|(dpt_mean_ratio>2.5)).astype(int)

df = pd.DataFrame({
    'montant':montant, 'depense_weekend':depense_weekend,
    'distance_km':distance, 'historique_anomalies':historique,
    'ratio_moyenne_departement':dpt_mean_ratio,
    'alerte_declenchee':depassement_seuil, 'is_fraud':is_fraud})

# 2. PREPROCESSING
X = df.drop('is_fraud', axis=1); y = df['is_fraud']
X_train,X_test,y_train,y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# 3. LOGISTIC REGRESSION
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
y_prob_lr = lr.predict_proba(X_test_scaled)[:,1]

# 4. RANDOM FOREST
rf = RandomForestClassifier(n_estimators=100, random_state=42,
                            class_weight='balanced')
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
y_prob_rf  = rf.predict_proba(X_test_scaled)[:,1]

# 5. ISOLATION FOREST
iso = IsolationForest(contamination=df['is_fraud'].sum()/len(df),
                      random_state=42)
iso.fit(X_train_scaled)
y_pred_iso = np.where(iso.predict(X_test_scaled)==-1, 1, 0)
y_prob_iso = -iso.score_samples(X_test_scaled)

# 6. COURBES ROC (roc_curves.png)
plt.figure(figsize=(8,6))
for y_prob, name in [(y_prob_lr,'LR'),(y_prob_rf,'RF'),(y_prob_iso,'IsoForest')]:
    fpr,tpr,_ = roc_curve(y_test, y_prob)
```

```
    plt.plot(fpr, tpr, label=f'{name} (AUC={roc_auc_score(y_test,y_prob):.3f})')
plt.plot([0,1],[0,1],'k--'); plt.legend(); plt.savefig('roc_curves.png')
```